

Plant X

Done by

AISWARYA P DAS 200021089961

Under the guidance of

Ms NIBY BABU

*In partial fulfillment of requirements for
the award of the Degree of Bachelor
of Computer Applications*

Of

Mahatma Gandhi University ,Kottayam, Kerala



Department of Computer Science

NIRMALA COLLEGE, MUVATTUPUZHA

(Affiliated to Mahatma Gandhi University)

2020-2023

NIRMALA COLLEGE
(Affiliated to M.G.University,Kottayam)
MUVATTUPUZHA



CERTIFICATE

*Certified that this is a bonafide report on the project work entitled
..... done
byReg.No
during the year in the partial fulfillment of the requirements for the
award of the degree of Bachelor of Computer Applications of Mahatma Gandhi
University, Kottayam, Kerala*

Internal Guide

Head of the Department

Submitted for the Viva-voce Examination held on.....

External Examiner

ACKNOWLEDGEMENT

I give all honor and praise to the **Lord** who gave wisdom and enable to complete my project successfully.

I express my sincere and heartfelt thanks to respected **Dr.Thomas K.V**, our beloved principal for providing necessary facilities for the completion of our project successfully.

I think profusely **Ms. Preethy George**, Course Coordinator of Computer Science Department for her guidance and inspiration throughout our course of study.

I express my sincere gratitude to my project guide, **Ms Niby Babu** Assistant Professor in Computer Science Department for the valuable advices and guidance throughout the completion of my project.

I also express my gratitude and thanks to all my teachers and friends for their sincere and friendly cooperation in the successful completion of my project.

OBJECTIVE AND SCOPE

To design such system that can detect crop disease and pest accurately. Create database of insecticides for respective pest and disease. To provide remedy for the disease that is detected. Plant diseases and pests detection is a very important research content in the field of machine vision. It is a technology that uses machine vision equipment to acquire images to judge whether there are diseases and pests in the collected plant images. The detection of plant disease is of vital importance in practical agricultural production. It scrutinizes the plant's growth and health condition and guarantees the regular operation and harvest of the agricultural planting to proceed successfully. The study of plant diseases is important as they cause loss to the plant as well as plant produce. The various types of losses occur in the field, in storage or any time between sowing and consumption of produce. The diseases are responsible for direct monetary loss and material loss.

CONTENTS

1.Introduction	1
2.System analysis	2
2.1 Existing system	2
2.2 Proposed system	3
2.3 Software requirement specification	3
2.3.1 Hardware specification	4
2.3.2 Software specification	4
2.3.3 Front End	5
2.3.4 Back End	6
2.4 Feasibility Analysis	6
3.System Design	15
3.1 Module Description	--
3.2 Input Design	15
3.3 Database Design	--
3.4 Output Design	16
3.5 Dataflow Diagram	16
4.System Testing and Implementation	22
4.1 System Testing	22
4.2 System Implementation	25
5.Maintenance	28
6.Scope for Future Enhancement	30
7.Conclusion	31
8.Bibliography	32
9.Appendix	33
11.1 Screenshots	33
11.2 Codes	--

INTRODUCTION

1.INTRODUCTION

My project entitled with 'Plant X' a plant disease detection system. Many plant diseases cause less dramatic losses annually throughout the world but collectively constitute sizable losses to farmers and can reduce the aesthetic values of landscape plants and home gardens. The goal of plant disease management is to reduce the economic and aesthetic damage caused by plant diseases. Plant analysis has proven useful in confirming nutrient deficiencies, toxicities or imbalances, identifying "hidden hunger," evaluating fertilizer programs, determining the availability of elements not tested for by other methods, and studying interactions among nutrients.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

System analysis is a step-by-step process used to identify and develop or acquire the software need to control the processing of specific application. System analysis is a continuing activity the stages of the systems development. System analysis is the process of gathering and interpreting facts, diagnosing problems and using the facts to improve the system. The outputs from the organization are traced through the various processing that the input phases through in the organization. This involves gathering information and using structured tools for analysis. A detailed study of this process must be made by various techniques like interviews, questionnaires etc.

It is necessary to have such a good system analysis and then by a project development cycle so that the project can be completed in a strictly manner and able to finish with the desired time. The analyst must be so careful about his responsibilities.

2.1 EXISTING SYSTEM

The existing system is very limited and is fully manual

Limitation of existing system

- Users want to go detection center for disease detection
- Taking too much time for getting results

2.2 PROPOSED SYSTEM

The proposed system simplifies the manual process of handling in events and buying artworks

- The system is more flexible and convenient to use.
- All working are manually done
- Completely automated
- Admin can view previous history
- High processing speed and high classification accuracy.

2.3 SYSTEM REQUIREMENT SPECIFICATION

A software requirements specification (SRS) is a comprehensive description of the intended purpose and environment for software under development. The SRS fully describes what the software will do and how it will be expected to perform. An SRS minimizes the time and effort required by developers to achieve desired goals and also minimizes the development cost. A good SRS defines how an application will interact with system hardware, other programs and human users in a wide variety of real-worked situations.

Customer requirements

- The system should be fast
- User friendly
- Maintaining security of data
- Efficiency in data retrieval and management

2.3.1 Hardware Specifications

Processor	: INTEL Core i3
Speed	: 3.0GHz or higher
System bus	: 64bits
Memory	: 8GB RAM
Hard disk	: 512GB
Monitor	: 15.6" LCD Monitor
Keyboard	: 108 keys

2.3.2 Software Specifications

Operating System	: Windows
Front End	: Django
Back End	: sqlite3
Designing tools	: HTML , CSS
Browser	: Google Chrome

2.3.3 Front end

Django

Django is a high-level web framework written in Python that allows developers to rapidly develop web applications. It follows the Model-View-Controller (MVC) architectural pattern and provides a clean and pragmatic approach to web development. Django's popularity is largely due to its “batteries included” philosophy, which means it comes with a wide range of built-in features and functionalities that can help developers create complex web applications with ease.

These features include a powerful ORM (Object-Relational Mapping) for database access, a templating engine for rendering HTML templates, and a robust set of security features to protect web applications from common attacks such as cross-site scripting (XSS) and SQL injection. Django also has a strong and active community that provides extensive documentation and support, making it easier for developers to get started and troubleshoot issues. Overall, Django is a powerful and versatile web framework that enables developers to build scalable and secure web applications quickly and efficiently.

HTML

The **HyperText Markup Language** or **HTML** is the standard markup language for documents designed to be displayed in a web browser. It can be assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.

HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered page. HTML provides a means to create structured documents by denoting structural semantics for text such as headings, paragraphs, lists, links, quotes and other items. HTML elements are delineated by *tags*, written using angle brackets.

2.1.1 Back end

SQLite3

SQLite3 is a lightweight, serverless, and self-contained relational database management system That is widely used in embedded systems, mobile applications, and desktop applications. It is a popular choice for developers who need to store data locally within an application without requiring a separate database server. SQLite3 stores data in a single file making it easy to transport and backup. It supports most of the SQL92 standard and provides a range of features such as transactions, triggers, and views, which allow developers to create and manage complex databases. SQLite3 is also highly scalable and can handle databases with millions of rows efficiently. Another advantage of SQLite3 is its cross-platform compatibility, which means it can be used with multiple programming languages and operating systems. Overall, SQLite3 is a reliable and flexible databases system that offers a convenient and lightweight solution for managing local data in applications.

2.1 FEASIBILITY ANALYSIS

A feasibility study is an evaluation and analysis of the potential of the proposed project which is based on extensive investigation and research to give full comfort to the decision makers. Feasibility studies aim to objectively and rationally uncover the strength and weakness of existing business of proposed venture, opportunities and threads as presented by the environment, the resources required to carry through, and ultimately the process for success. In its simplest terms, the two criteria to judge feasibility are cost required and value to attain. As such, a well-designed feasibility study should provide a historical background of the business or project, description of the product or service, accounting statements, details of the operations and management, marketing research and policies, financial data, legal requirements and tax obligations.

The two aspects in the feasibility study are:

- Technical feasibility
- Operational feasibility

Technical Feasibility

The technical feasibility centres on the existing system and what extend it can support the proposed addition. The technical feasibility assessment is focused on gaining an understanding of the present technical resources of the organization and their applicability to the expected needs of the proposed system. The minimum requirements of the system are met by average user. The developer system has á modest technical requirement as only minimal or null changes are required for implementing system.

Normally associated with the technical feasibility includes:

- Development risk
- Resource availability
- Technology

The proposed system can work without any additional hardware or software support other than the computer system and networks. So, I analysed that the proposed system is much more technically feasible than other systems when comparing with the benefits of the new system.

Operational Feasibility

Operational feasibility is a measure of how well a proposed system solves the problems, and takes advantage of the opportunities identified during scope definition and how it satisfies the requirements identified in the requirements analysis phase of system development.

SYSTEM DESIGN

3. SYSTEM DESIGN

3.1 MODULE DESCRIPTION

Admin

Admin is the one who can view the users details. He can view the history of users plant detection, and he can sort out the monthly reports and also he can view the feedbacks given by the users. This is the roles of admin in this project.

User

User can register and login. And for plant disease detection they can upload their plant pic and the results get very quickly. Also the user can update their profile by login the application. After the result they can receive a result report and can download it. Users can write their feedbacks of the application.

3.2 INPUT DESIGN

The quality of the system input determines the quality of the system output. Input specification describes the manner in which data enter the system for processing. Input design features can ensure the reliability of the system and produce result from accurate data, or they can result in the production of erroneous information. The input design also determines whether the user can interact efficiently with the system.

In our system almost, all inputs are being taken from the databases. To provide adequate inputs we have to select necessary values from the databases and arrange it to the appropriate controls.

DNN ALGORITHM

Deep neural network (DNN) models can address these limitations of matrix factorization. DNNs can easily incorporate query features and item features (due to flexibility of the input layer of the network), which can help capture the specific interests of a user and improve the relevance of recommendations. A neural network with some level of complexity, usually at least two layers, qualifies as a deep neural network(DNN) or deep net for short

3.3 DATABASE DESIGN

Table name: User

Description: This tables stores the user details

Primary key: u id

Foreign key :user

Field	Type	Size	Discription
name	varchar	50	Name of the user
email	varchar	50	Email of the user
phone	int	10	Phone number of the user
address	varchar	50	Address of the user
district	varchar	50	District of the user
pin	int	6	Pin number of the user
user	varchar	20	Username of the user

Table : FeedbackSite

Description: table to add user feedback

Primary key:

Foreign key:

Field	Type	Size	Discription
user	varchar	20	Username of the user
feedback	varchar	50	Feedback of user
date	date	10	Date of the feedback entered

Table : ImageDetection

Description: table to add user detection

Primary key:

Foreign key:

Field	Type	Size	Description
user	varchar	20	User of the user
date	date	20	Date of the detection
image			Image for detection
result	varchar	50	Result of detection

3.4 OUTPUT DESIGN

One of the important features of an information system for users is the output produces. Output is the information delivered to users through the information system. Without quality of the output, the entire system appears to be unnecessary that users will avoid using it. Users generally merit the system solely by its output. In order to create the most useful output possible. One works closely with the user though an interactive process. until the result is considered to be satisfactory.

3.5 DATA FLOW DIAGRAM (DFD)

Introduction to data flow diagram

A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system. It differs from the flowchart as it shows the data flow instead of the control flow of the program. A data flow diagram can also be used for the visualization of data processing (structured design).

Data flow diagrams were invented by Larry Constantine, the original developer of structured design, based on Martin and Estrin's "data flow graph" model of computation.

Data flow diagrams (DFDs) are one of the three essential perspectives of Structured System Analysis and Design Method SSADM. The sponsor of a project and the end users will need to be briefed and consulted throughout all stages of a system's evolution. With a data flow diagram, users are able to visualize how the system will operate, what the system will accomplish, and how the system will be implemented. The old system's data flow diagrams can be drawn up and compared with the new system's data flow diagrams to draw comparisons to implement a more efficient system. Data flow diagrams can be used to provide the end user with physical idea of where the data they input ultimately has an effect upon the structure of the whole system from order to dispatch to report. How any system is developed can be determined through a data flow diagram.

Developing a data flow diagram helps in identifying the transaction data in the data model. There are different notations to draw data flow diagrams, defining different visual representation for process, data stores, data flow, and external entities. The first step is to draw a data flow diagram (DFD). A DFD also known as "bubble chart" has the purpose of clarifying system requirements and identifying major transformation that will become program in system design. So, it is starting point of the design phase that functionally decompose the requirements specification down to the lowest level of details. DFD consists of series of bubbles joined by lines. The bubbles represent data transformation and the lines represent data flow in the system.

DFD Symbols

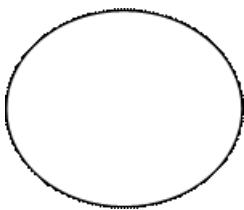
Square- Defines source or destination of system.



Data flow - Identifies data flow Circle



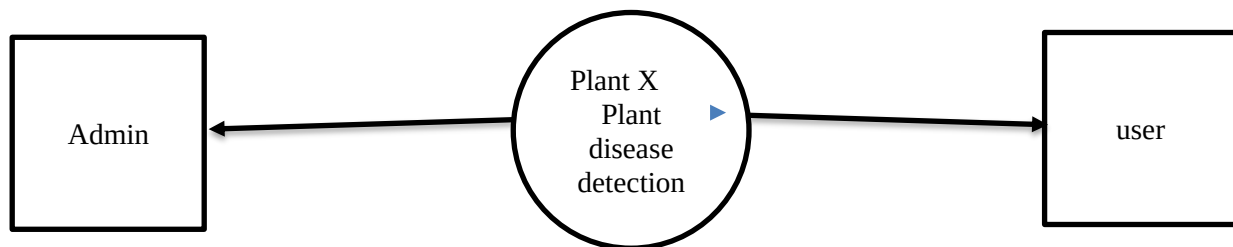
Bubble - Represents a process that transforms incoming data to outgoing data.



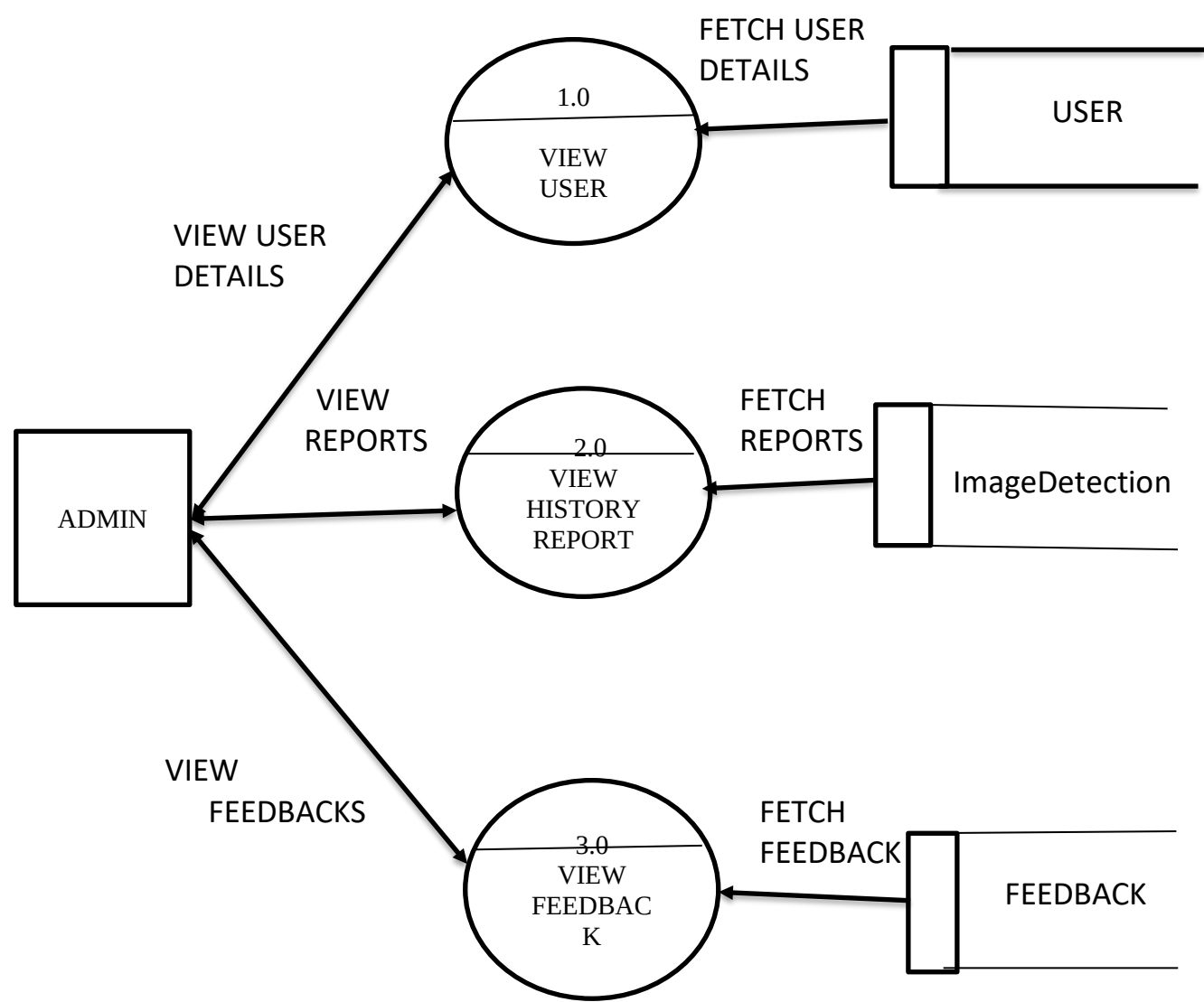
Open rectangle- Data store



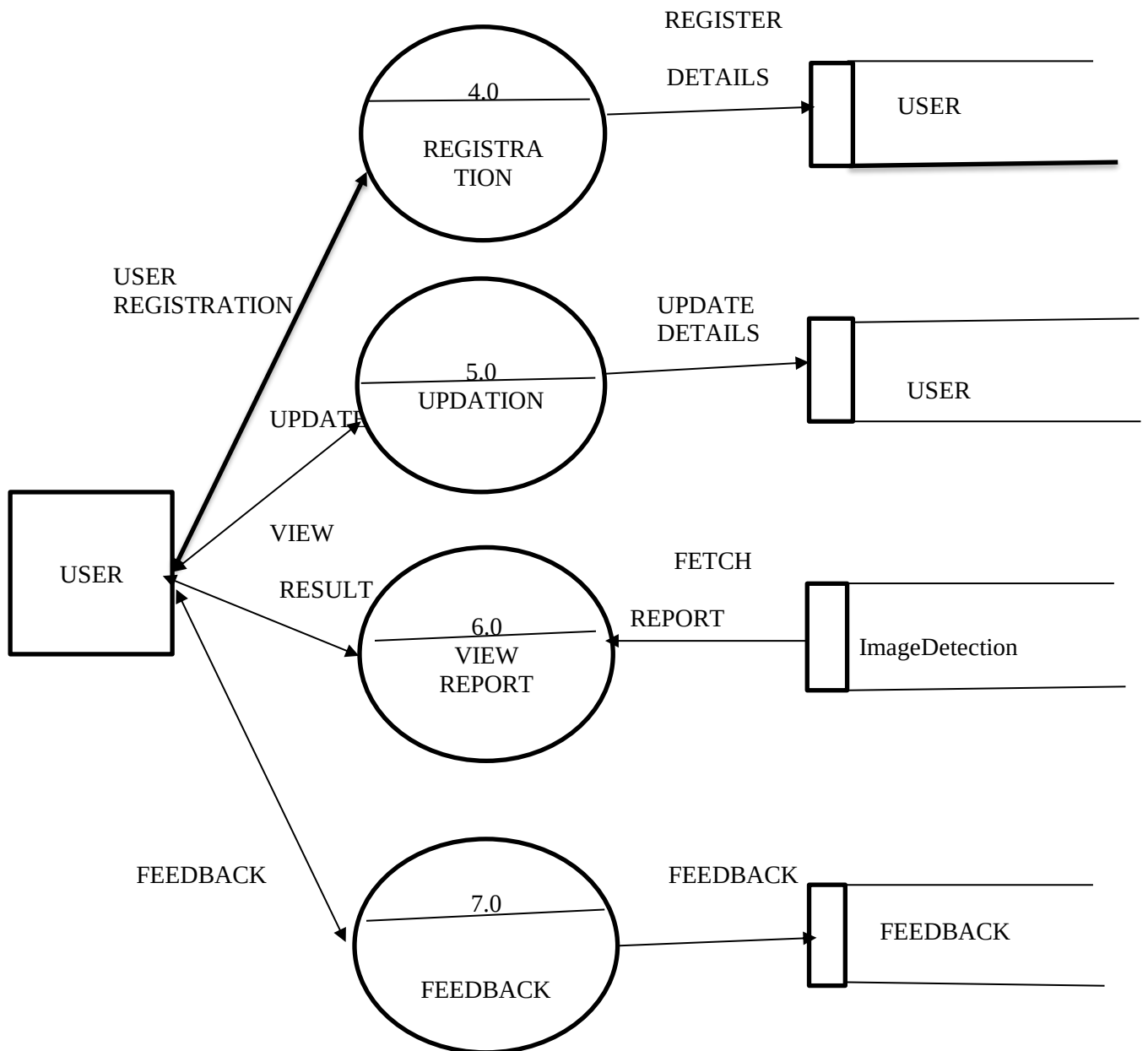
Level-0



level 1-- Admin



LEVEL 1- USER



SYSTEM TESTING

4.SYSTEM TESTING

4.1 SYSTEM TESTING

Testing is the process of examining the software to compare the actual behaviour with that of the expected behavior. The major goal of software testing is to demonstrate that faults are not present. In order to achieve this goal, the tester executes the program with the intent of finding errors. Though testing cannot show absence of errors but by not showing their presence it is considered that these are not present.

System testing is defined as the process by which one detects the defects in the software. Any software development organization or team has to perform several processes. Software testing is one among them. It is the final opportunity of any programmer to detect and rectify any defects that may have appeared during the software development stage. Testing is a process of testing a program with the explicit intention of finding errors that makes the program fail. In short system testing and quality assurance is a review in software products and related documentation for completion, correctness, reliability and maintainability.

System testing is the first stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operation commences. Testing is vital to the success of the system. System testing makes a logical assumption that if all the parts of the system are correct and the goal will be successfully achieved. A series of testing are performed for the proposed system before the proposed system is ready for user acceptance testing.

The testing steps are,

- Unit Testing
- Integration Testing
- Validation testing
- Output Testing
- Acceptance Testing

System Testing provides the file assurance that software once validated must combined with all other system elements. System testing verifies whether all elements have been combined properly and that overall system function and performance is achieved. For the integration of modules, the validation test

was carried out over the system. It was that all the modules work well together and meet the overall system function and performance.

Unit Testing

Unit testing is carried out screen-wise, each screen being identified as an object. Attention is diverted to individual modules, independently to one another to locate errors. This has enabled the detection of errors in coding and logic.

Various test cases are prepared. For each module these test cases are implemented and it is checked whether the module is executed as per the requirements and outputs the desired result. In this test each service input and output parameters are checked.

In unit testing:

- Module interface was tested to ensure that information properly flows into and out of the program under test.
- Boundary condition was tested to ensure that module operates properly at boundaries established to limit or restrict processing.
- All independent paths through the control structures were executed to ensure that all statements in the modules have been executed at least once.
- Error handling paths were also tested.

Integration Testing

Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing.

Unit tested module were taken and a single program structure was built that has been dictated by the design. Incremental integration has been adopted here.

The modules are tested separately for accuracy and modules are integrated too. It is using bottom up integration i.e., by integrating from moving from bottom to the top the system is checked and errors found during integration are rectified. In this testing individual modules were combined and the module wise shifting was verified to be alright.

The entire software was developed and tested in small segments, where errors were easy to locate and rectify. Program builds (group of modules) were constructed corresponding to the successful testing of user interaction, data manipulation analysis, and display processing and database management.

Validation Testing

Validation testing is done to ensure complete assembly of the error-free software. Validation can be termed successful only if it functions in manner. Reasonably expected by the student under validation is alpha and beta testing. The student-side validation is done in this testing phase. It is checked whether the data passed to each student is valid or not. Entering incorrect values does the validation testing and it is checked whether the errors are being considered. Incorrect values are to be discarded. The errors are rectified.

In "University result portal" verifications are done correctly. So, there is no chance for users to enter incorrect values. It will give error messages by using different validations. The validation testing is done very clearly and found it is error free.

Output Testing

After performing the validation testing the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in a specific format.

The output format on the screen was found to be correct as the format was designed in the system design phase according to the user needs. For the hard copy also, the output comes out as specified requirement by the user. Hence output testing does not result in any Correction in the system. output This project is developed based on the user choice. It is user friendly. The output format is very clear to user. Output testing is done on Smart builders correctly.

Acceptance testing

Acceptance involves running a suite of tests on the completed system. Each individual test, known as a Case, exercise particular operating condition of the operating condition of the user's environment or feature of the system, and will result in a pass fail, or Boolean outcome.

5. **SYSTEM IMPLEMENTATION**

The implementation is the final state and it is an important phase. It involves the invalid programming system testing, user training and the operational running of developed proposed system that constitutes the application subsystems. A major task of preparing for implementation is education of users, which should really have been taken place much earlier in the project when they were being involved in the investigation and design work. During the implementation phase system actually takes physical shape. In order to develop a system implemented planning is very essential.

The implementation phase of the software development is concerned with translating design specification into source code. The user tests the developed system and changes are made according to their needs. Our system has been successfully implemented.

Before implementation several tests have been conducted to ensure that no errors are encountered during the operation. The implementation phase ends with an evaluation of the system after placing into the operation for a period of time.

The process of putting the developed system in actual use is called system implementation. This includes all those activities that take place to convert from old system to new system. The system can be implemented only after testing is done and is found to be working to specifications. The implementation stage is a systems project in its own right.

The implementation stage involves following tasks:

- Careful planning.
- Investigation of system and constraints.
- Design of method to achieve change over
- Evaluation of the changeover method.

In the case of this project all the screens are designed first. For making it to be executable, codes are written on each screen and performs the implementation by creating the database and connecting to the server. After that the system, is Checked, whether it performs all the transactions Correctly. Then databases are cleared and made it to be usable to the technicians.

MAINTENANCE

6. MAINTENANCE

Software maintenance is the modification of a software product after delivery to correct faults, to improve performance or other attributes. Maintenance is the ease with which a program can be corrected if any error is encountered, adapted if its environment changes or enhanced if the customer desires a change in requirement. Maintenance follows conversation to extend that changes are necessary to maintain satisfactory operations relative to changes in the user's environment.

Maintenance often includes minor enhancements or corrections to problems that surface in the system's operation. Maintenance is also done based on fixing the problems reported, changing the interface with other software or hardware enhancing the software.

CATEGORIES OF MAINTENANCE

Corrective Maintenance

Corrective maintenance is the most commonly used maintenance approach, but it is easy to see its limitations. When equipment fails, it often leads to downtime in production, and sometimes damages other parts. In most cases, this is expensive. Also, if the equipment needs to be replaced, the cost of replacing it alone can be substantial. Reliability of systems maintained by this type of maintenance is unknown and cannot be measured. Corrective maintenance is possible since the consequences of failure or wearing out are not significant and the cost of this maintenance is not great

Adaptive Maintenance

Modification of a software product performed after delivery to keep a software product usable in a changed or changing environment. Adaptive maintenance includes any work initiated as a consequence of moving the software to a different hardware or software platform. It is a change driven by the need to accommodate modifications in the environment of software system. The environment in this context refers to the totality of all conditions and influences which act from outside upon the system. A change to the whole or part of this environment will warrant a corresponding modification of the software.

Perfective Maintenance

Modification of a software product after delivery to improve performance or maintainability. This term is used to describe changes undertaken to expand the existing requirements of the system. A successful piece of software tends to be subjected to a succession of changes resulting in an increase in its requirements. This is based on a premise that as the software becomes useful, the user experiments with new cases beyond the scope for which it was initially developed. Expansion requirements can take the form of enhancement of existing system functionality and improvement in computational efficiency.

Preventive Maintenance

Preventive maintenance is a schedule of planned maintenance actions aimed at the prevention of breakdowns and failures. The primary goal of preventive maintenance is to prevent the failure of equipment before it actually occurs. It is designed to preserve and enhance equipment reliability by replacing worn components before they actually fail. Preventive maintenance activities include equipment checks, partial or complete overhauls at specified periods.

Long-term benefits of preventive maintenance include:

- Improved system reliability.
- Decreased cost of replacement.
- Decreased system downtime.

SCOPE FOR FUTURE ENHANCEMENT

7.SCOPE FOR FUTURE ENHANCEMENT

The drawbacks of the existing system as listed before are fully evacuated. All the existing inconsistencies are fully solved as this system is implemented.

- All users can take live pic and detect the diseases.
- An online bill payment system
- Rating, filter based search

CONCLUSION

8. CONCLUSION

Early diagnosis of plant diseases using accurate or automatic detection technique can enhance the quality of food production and minimize economic losses. In recent years, deep learning has brought tremendous improvements in the recognition accuracy of image classification and object detection systems. A plant diseases is defined as “anything that prevents a plant from performing to its maximum potential.” This definition is broad and includes abiotic and biotic plant diseases. Abiotic or non-infectious diseases. These diseases are caused by con-ditions external to the plant, not living agents.

BIBLIOGRAPHY

9.BIBLIOGRAPHY

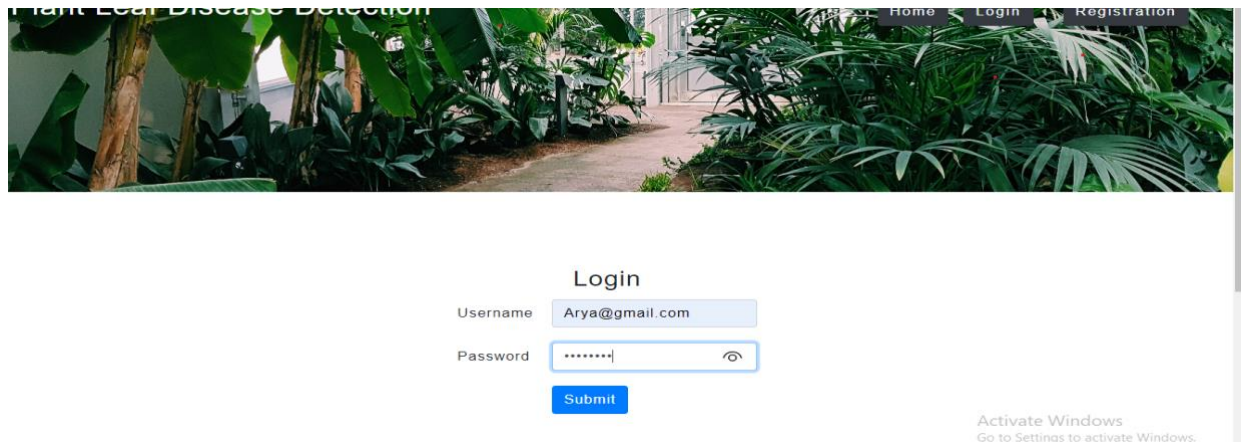
1. Jalotte, Pankaj. " **SOFTWARE ENGINEERING**", Third Edition, Spring Publishers, Narosa Publishing House.
2. Ramakrishnan, Raghu. and_ Gehrke, Johannes. **DATABASE MANAGEMENT SYSTEM**", Third Edition, McGraw-Hill Professional Publication.
3. Awad, Elias M. "**SYSTEM ANALYSIS AND DESIGN**", Second Edition, McGraw-Hill Professional publication.
4. Rob, Peter. and Coronel, Carlos. "**DATABASE SYSTEMS DESIGN ANDMANAGEMENT**", Fifth Edition, Thomson Learning Publication.
5. Pressman, Roger S. "SOFTWARE ENGINEERING", Fifth Edition, McGraw-Hill Publication.
6. Bayross, Ivan. "**SQL/MySQL**" Second Revised Edition, BPB Publications

APPENDIX

10. APPENDIX

9.1 SCREEN SHOTS

Login page



Plant Leaf Disease Detection

Home Login Registration

Login

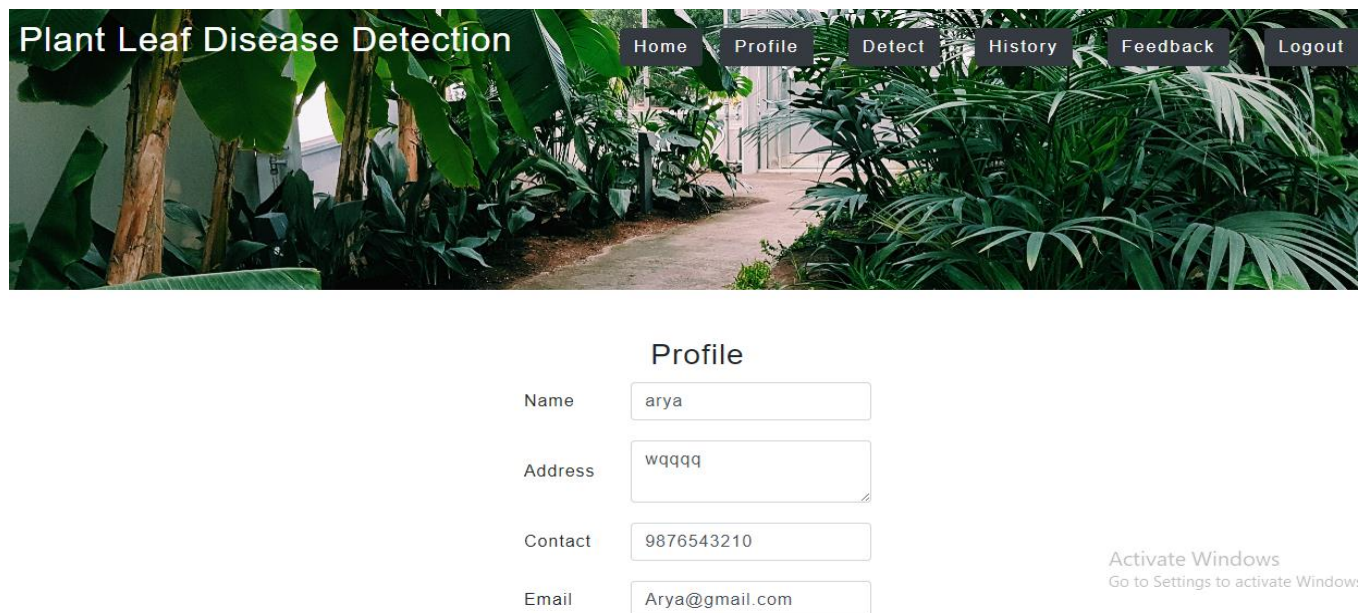
Username Arya@gmail.com

Password *****

Submit

Activate Windows
Go to Settings to activate Windows.

USER REGISTRATION



Plant Leaf Disease Detection

Home Profile Detect History Feedback Logout

Profile

Name arya

Address wqqqq

Contact 9876543210

Email Arya@gmail.com

Activate Windows
Go to Settings to activate Windows.

IMAGE DETECTION

Plant Leaf Disease Detection

Home Profile Detect History Feedback Logout

Detect Image

Choose File image_2023...4939956.png

Submit

DETECTION RESULT

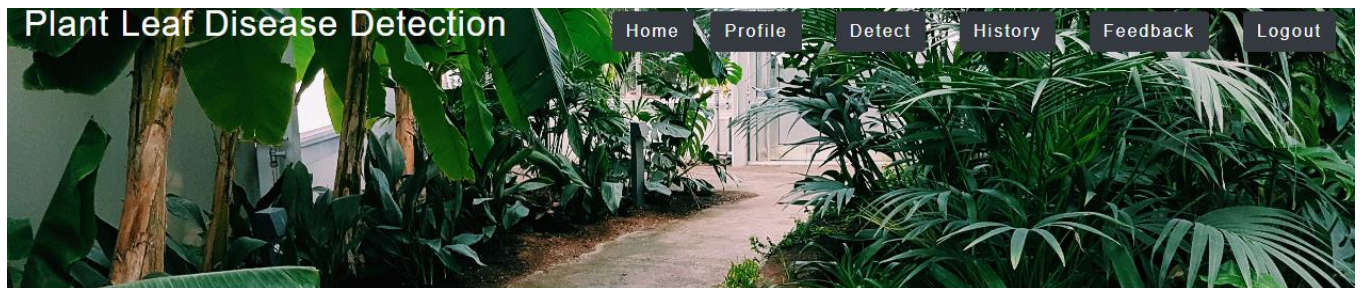
Plant Leaf Disease Detection

Home Profile Detect History Feedback Logout

Result:-
Status:
Unhealthy.
Disease:
Bacterial.

Activate Windows
Go to Settings to activate Windows.

FEEDBACK



Feedback

GOOD

Submit

9.2 CODES

Login page

```
{% extends 'commonbase.html' %}
{% block content %}
<style>
    td,th{
        padding: 10px;
    }
    table{
        margin:auto
    }
</style>
<div style="margin: 100px;">
    <form method="POST">
        {% csrf_token %}
        <h3 style="text-align:center;">Login</h3>
        <table>
            <tr>
                <td>Username</td>
                <td><input type="text" class="form-control" name="txtUname" required /></td>
            </tr>
            <tr>
                <td>Password</td>
                <td><input type="password" class="form-control" name="txtPwd" required /></td>
            </tr>
            <tr>
                <td></td>
                <td><input type="submit" class="btn btn-primary" name="btnSubmit" required /></td>
            </tr>
        </table>
    </form>
</div>
<script>
    var msg="{{msg}}";
    if(msg!="")
        alert(msg);

</script>
{% endblock %}
```

REGISTRATION

```
{% extends 'userbase.html' %} {% block content %}
<style type="text/css">
    #log {
        background-color: rgba(0, 150, 220, 0.5);
        margin: 10px 150px;
        padding: 50px;
        color: white;
        width: 1000px;
        float: left;
    }
    td,
    th {
        padding: 10px;
    }
    #tbl {
        width: 900px;
    }
    a {
        color: #ebd231;
    }
    table {
        border-collapse: collapse;
        border: none;
    }
    th {
        background-color: rgba(0, 100, 180, 0.7);
        color: #ebd231;
    }
    /*tr:nth-child(odd) {background-color: silver;}*/
</style>
<div class="mt-5 mb-5" style="width: 100%">
    <form method="POST">
        {% csrf_token %}
        <h3 style="text-align: center">Profile</h3>
        <table style="margin: auto">
            <tr>
                <td>Name</td>
                <td>
                    <input
                        type="text"
                        class="form-control"
                        name="txtName"
                        pattern="[a-zA-Z ]+"

```

```

                                value="{{data.name}}"
                                required
                            />
                        </td>
                    </tr>
                    <tr>
                        <td>Address</td>
                        <td>
                            <textarea class="form-control" name="txtAddress" required>
{{data.address}}</textarea>
                        </td>
                    </tr>
                    <tr>
                        <td>Contact</td>
                        <td>
                            <input
                                type="text"
                                class="form-control"
                                name="txtContact"
                                value="{{data.phone}}"
                                pattern="[6789][0-9]{9}"
                                required
                            />
                        </td>
                    </tr>
                    <tr>
                        <td>Email</td>
                        <td>
                            <input
                                type="email"
                                class="form-control"
                                name="txtEmail"
                                value="{{data.email}}"
                                required
                            />
                        </td>
                    </tr>
                    <tr>
                        <td>Password</td>
                        <td>
                            <input
                                type="password"
                                class="form-control"
                                name="txtPwd"
                                pattern="^(?=.*[A-Za-z])(?=.*\d)(?=.*[@$!%*#?&])[A-Za-
z\d@$!%*#?&]{8,}$"
                                required
                            />
                        </td>
                    </tr>
                </tr>
            </tr>

```

```

        <td></td>
        <td>
            <input
                type="submit"
                class="btn btn-primary"
                value="Update"
                name="btnSubmit"
                required
            />
        </td>
    </tr>
</table>
</form>
</div>
<script>
    var msg = '{{msg}}';
    if (msg != "") {
        alert(msg);
        window.location = '/customerprofile';
    }
</script>
{% endblock %}

```

IMAGE DETECTION

```

{% extends 'userbase.html' %} {% block content %}
<style type="text/css">
    #log {
        background-color: rgba(0, 150, 220, 0.5);
        margin: 10px 150px;
        padding: 50px;
        color: white;
        width: 1000px;
        float: left;
    }
    td,
    th {
        padding: 10px;
    }
    #tbl {
        width: 900px;
    }
    a {
        color: #ebd231;
    }
    table {
        border-collapse: collapse;
        border: none;

```

```

    }

    th {
        background-color: rgba(0, 100, 180, 0.7);
        color: #ebd231;
    }
    .result{
border-radius: 50px;
background: #fafafa;
box-shadow: 20px 20px 39px #d5d5d5,
            -20px -20px 39px #ffffff;
width: 300px;
height:300px;
display: flex;
justify-content: center;
align-items: center;
margin: auto;
margin-top: 20px;
margin-bottom: 50px;
    }
    /*tr:nth-child(odd) {background-color: silver;}*/
</style>
<div class="mt-5 mb-5" style="width: 100%">
    {% if data %}
        <div class="result">
            <h1 class="text-center"><span class="text-warning"> Result:-
</span><br>{{ data }}</h1>
        </div>

    {% endif %}
    <form method="POST" enctype="multipart/form-data">
        {% csrf_token %}
        <h3 style="text-align: center">Detect Image</h3>
        <table style="margin: auto">
            <td colspan="2">
                <input type="file" class="form-control" name="img" required>
            </td>
        </tr>
        <tr>
            <td></td>
            <td>
                <input
                    type="submit"
                    class="btn btn-primary"
                    value="Submit"
                    name="btnSubmit"
                    required
                />
            </td>
        </tr>
    </table>

```

```

    </form>
</div>
<script>
    var msg = '{{msg}}';
    if (msg != "") {
        alert(msg);
    }
</script>
{% endblock %}

```

FEEDBACK

```

{% extends 'userbase.html' %} {% block content %}
<style type="text/css">
    #log {
        background-color: rgba(0, 150, 220, 0.5);
        margin: 10px 150px;
        padding: 50px;
        color: white;
        width: 1000px;
        float: left;
    }
    td,
    th {
        padding: 10px;
    }
    #tbl {
        width: 900px;
    }
    a {
        color: #ebd231;
    }
    table {
        border-collapse: collapse;
        border: none;
    }

    th {
        background-color: rgba(0, 100, 180, 0.7);
        color: #ebd231;
    }
    /*tr:nth-child(odd) {background-color: silver;}*/
</style>
<div class="mt-5 mb-5" style="width: 100%">
    <form method="POST">
        {% csrf_token %}
        <h3 style="text-align: center">Feedback</h3>
        <table style="margin: auto">
            <td colspan="2">

```

```

        <textarea class="form-control" name="feedback" required></textarea>
    >
    </td>
</tr>
<tr>
    <td></td>
    <td>
        <input
            type="submit"
            class="btn btn-primary"
            value="Submit"
            name="btnSubmit"
            required
        />
    </td>
</tr>
</table>
</form>
</div>
<script>
    var msg = '{{msg}}';
    if (msg != "") {
        alert(msg);
    }
</script>
{% endblock %}

```

USER HISTORY

```

{% extends 'userbase.html' %} {% block content %}
<style type="text/css">
    #log {
        background-color: rgba(0, 150, 220, 0.5);
        margin: 10px 150px;
        padding: 50px;
        color: white;
        width: 1000px;
        float: left;
    }
    td,
    th {
        padding: 10px;
    }
    #tbl {
        width: 900px;
    }
    a {
        color: #ebd231;

```

```

    }
    table {
        border-collapse: collapse;
        border: none;
    }

    th {
        background-color: rgba(12, 180, 0, 0.7);
        color: #ebd231;
    }
    /*tr:nth-child(odd) {background-color: silver;}*/
</style>
<div style="margin: 100px">
    <form method="POST">
        {% csrf_token %}
        <h3 style="margin: 20px">Detection History</h3>
        <table border="0" style="margin: 50px">
            <tr>
                <th>DATE</th>
                <th>Image</th>
                <th>Result</th>
            </tr>
            {% for d in data %}
            <tr>
                <td>{{d.date}}</td>
                <td>
                    <a href="/static/media/{{d.image}}" target="_blank"
                    ></a>
                </td>
                <td>{{d.result}}</td>
            </tr>
            {% endfor %}
        </table>
    </form>
</div>
{% endblock %}

{% extends 'adminbase.html' %} {% block content %}
<style type="text/css">
    #log {
        background-color: rgba(0, 150, 220, 0.5);
        margin: 10px 150px;
        padding: 50px;
        color: white;
        width: 1000px;
        float: left;
    }
    td,
    th {
        padding: 10px;

```

```

    }
    #tbl {
        width: 900px;
    }
    a {
        color: #ebd231;
    }
    table {
        border-collapse: collapse;
        border: none;
    }

    th {
        background-color: rgba(12, 180, 0, 0.7);
        color: #ebd231;
    }
    /*tr:nth-child(odd) {background-color: silver;}*/
</style>
<div style="margin: 100px">
    <form method="POST">
        {% csrf_token %}
        <h3 style="margin: 20px">Detection History</h3>
        <table border="0" style="margin: 50px">
            <tr>
                <th>DATE</th>
                <th>User</th>
                <th>Image</th>
                <th>Result</th>
            </tr>
            {% for d in data %}
            <tr>
                <td>{{ d.date }}</td>
                <td>{{ d.user.name }}</td>
                <td>
                    <a href="/static/media/{{ d.image }}" target="_blank"
                        ></a>
                </td>
                <td>{{ d.result }}</td>
            </tr>
            {% endfor %}
        </table>
    </form>
</div>
{% endblock %}

```

